

13. Sean $U_i, i \geq 1$, números aleatorios. Defina N como

$$N = \text{Máximo} \left\{ n: \prod_{i=1}^n U_i \geq e^{-3} \right\}$$

donde $\prod_{i=1}^0 U_i \equiv 1$.

(a) Determine $E[N]$ por simulación.

(b) Determine $P\{N = i\}$, para $i = 0, 1, 2, 3, 4, 5, 6$, por simulación.

Bibliografía

- Knuth, D., *The Art of Computer Programming*, vol. 2, *Seminumerical Algorithms*, Reading, M. A., Addison-Wesley, 1981.
- L'Ecuyer, P., "Random Numbers for Simulation", en *Commun. Assoc. Comput. Mach.*, **33**, 1990.
- Marsaglia, G., "Random Numbers Fall Mainly in the Planes", en *Proc. Natl. Acad. Sci.*, **61**, 1962, pp. 25-28.
- Marsaglia, G., "The Structure of Linear Congruential Sequences", en S. K. Zaremba (comp.), *Applications of Number Theory to Numerical Analysis*, Londres, Academic Press, 1972, pp. 249-255.
- Naylor, T., *Computer Simulation Techniques*, Nueva York, Wiley, 1966.
- Ripley, B., *Stochastic Simulation*, Nueva York, Wiley, 1986.
- von Neumann, J., "Various Techniques Used in Connection with Random Digits, 'Monte Carlo Method'", en *U.S. National Bureau of Standards Applied Mathematics Series*, núm. 12, 1951, pp. 36-38.

CAPÍTULO

4

Generación de variables aleatorias discretas

4.1 El método de la transformada inversa

Suponga que queremos generar el valor de una variable aleatoria discreta X con función de masa de probabilidad

$$P\{X = x_j\} = p_j, \quad j = 0, 1, \dots, \sum_j p_j = 1$$

Para esto, generamos un número aleatorio U ; es decir, U está distribuido uniformemente en $(0, 1)$, y sea

$$X = \begin{cases} x_0 & \text{si } U < p_0 \\ x_1 & \text{si } p_0 \leq U < p_0 + p_1 \\ \vdots & \\ x_j & \text{si } \sum_{i=1}^{j-1} p_i \leq U < \sum_{i=1}^j p_i \\ \vdots & \end{cases}$$

Como $P\{a \leq U < b\} = b - a$ para $0 < a < b < 1$, tenemos que

$$P\{X = x_j\} = P\left\{\sum_{i=1}^{j-1} p_i \leq U < \sum_{i=1}^j p_i\right\} = p_j$$

y entonces X tiene la distribución deseada.

Observaciones

1. Podemos escribir lo anterior en forma algorítmica, como

Generar un número aleatorio U
 Si $U < p_0$ hacer $X = x_0$ y terminar
 Si $U < p_0 + p_1$ hacer $X = x_1$ y terminar
 Si $U < p_0 + p_1 + p_2$ hacer $X = x_2$ y terminar
 $\vdots$

2. Si los $x_i, i \geq 0$, están ordenados de modo que $x_0 < x_1 < x_2 < \dots$ y si F denota la función de distribución de X , entonces $F(x_i) = \sum_{j=0}^i p_j$ y entonces

$$X \text{ será igual a } x_j \quad \text{si} \quad F(x_{j-1}) \leq U < F(x_j)$$

En otras palabras, después de generar un número aleatorio U determinamos el valor de X hallando el intervalo $(F(x_{j-1}), F(x_j))$ en el que está U [o, de forma equivalente, hallando la inversa de $F(U)$]. Es por esta razón que el anterior se llama método de la transformada inversa discreta para generar X . ■

El tiempo necesario para generar una variable aleatoria discreta mediante este método es proporcional al número de intervalos en los que se realizan las búsquedas. Por esta razón, a veces vale la pena considerar los valores posibles x_j de X en orden decreciente de los p_j .

Ejemplo 4a Si queremos simular una variable aleatoria X tal que

$$p_1 = 0.20, \quad p_2 = 0.15, \quad p_3 = 0.25, \quad p_4 = 0.40 \quad \text{donde } p_j = P\{X = j\}$$

entonces podríamos generar U y hacer lo siguiente:

Si $U < 0.20$ hacemos $X = 1$ y terminamos
 Si $U < 0.35$ hacemos $X = 2$ y terminamos
 Si $U < 0.60$ hacemos $X = 3$ y terminamos
 En caso contrario, hacemos $X = 4$

Sin embargo, un procedimiento más eficaz es el siguiente:

Si $U < 0.40$ hacemos $X = 4$ y terminamos
 Si $U < 0.65$ hacemos $X = 3$ y terminamos
 Si $U < 0.85$ hacemos $X = 2$ y terminamos
 En caso contrario, hacemos $X = 1$ ■

Un caso en el que no es necesario buscar el intervalo adecuado en el cual se encuentra el número aleatorio es cuando la variable aleatoria deseada es la variable

aleatoria discreta uniforme. Es decir, suponga que queremos generar el valor de X entre los valores $1, \dots, n$, cada uno igualmente probable. Es decir, $P\{X = j\} = 1/n, j = 1, \dots, n$. El resultado anterior implica que podemos lograr esto al generar U y luego hacer

$$X = j \quad \text{si} \quad \frac{j-1}{n} \leq U < \frac{j}{n}$$

Por lo tanto, X será igual a j si $j-1 \leq nU < j$; o bien, en otras palabras,

$$X = \text{Ent}(nU) + 1$$

donde $\text{Ent}(x)$, algunas veces se denota como $[x]$, es la parte entera de x (es decir, el mayor entero menor o igual a x).

Las variables aleatorias discretas uniformes son importantes en la simulación, como lo indican los dos ejemplos siguientes.

Ejemplo 4b Generación de una permutación aleatoria. Suponga que estamos interesados en generar una permutación de los números $1, 2, \dots, n$ tal que todos los $n!$ posibles ordenamientos sean igualmente probables. Para ello, el siguiente algoritmo elige primero uno de los números $1, \dots, n$ al azar y luego lo coloca en la posición n ; enseguida, escoge al azar uno de los $n-1$ números restantes y lo sitúa en la posición $n-1$; entonces, elige al azar uno de los $n-2$ números restantes y lo coloca en la posición $n-2$, y así sucesivamente ("elegir un número al azar" significa que cada uno de los números restantes puede elegirse con la misma probabilidad). Sin embargo, para no tener que considerar con exactitud los números que faltan por colocar, es conveniente y eficiente mantenerlos en una lista ordenada y luego elegir al azar la posición del número y no el propio número. Es decir, comenzamos con cualquier ordenamiento inicial $P_1, P_2, \dots, P_n$ y elegimos una de las posiciones $1, \dots, n$ al azar, para luego intercambiar el número en esa posición con el de la posición n . Ahora elegimos al azar una de las posiciones $1, \dots, n-1$ e intercambiamos el número en esta posición con el número en la posición $n-1$, y así sucesivamente.

Si recordamos que $\text{Ent}(kU) + 1$ toma alguno de los valores $1, 2, \dots, k$, cada uno con la misma probabilidad, vemos que el algoritmo anterior para generar una permutación aleatoria se puede escribir como sigue:

PASO 1: Sea $P_1, P_2, \dots, P_n$ cualquier permutación de $1, 2, \dots, n$ (por ejemplo, podemos elegir $P_j = j, j = 1, \dots, n$).

PASO 2: Sea $k = n$.

PASO 3: Generar un número aleatorio U ; sea $I = \text{Ent}(kU) + 1$.

PASO 4: Intercambiar los valores de P_i y P_k .

PASO 5: Sea $k = k - 1$ y si $k > 1$, ir al paso 3.

PASO 6: $P_1, \dots, P_n$ es la permutación aleatoria deseada.

Por ejemplo, supongamos que $n = 4$ y que la permutación inicial es 1, 2, 3, 4. Si el primer valor de I (que puede ser 1, 2, 3 o 4 con la misma probabilidad) es $I = 3$, entonces se intercambian los elementos en las posiciones 3 y 4, de modo que la nueva permutación es 1, 2, 4, 3. Si el siguiente valor de I es $I = 2$, entonces se intercambian los elementos de las posiciones 2 y 3, de modo que la nueva permutación es 1, 4, 2, 3. Si el valor final de I es $I = 2$, entonces la permutación final es 1, 4, 2, 3, y éste es el valor de la permutación arbitraria.

Una propiedad muy importante del algoritmo anterior es que también se puede utilizar para generar un subconjunto aleatorio, digamos de tamaño r , de los enteros $1, \dots, n$. Basta seguir el algoritmo hasta llenar las posiciones $n, n-1, \dots, n-r+1$. Los elementos de estas posiciones son el subconjunto aleatorio (al hacerlo, siempre suponemos que $r \leq n/2$; ya que si $r > n/2$, entonces podríamos elegir un subconjunto aleatorio de tamaño $n-r$ y hacer que los elementos que no estén en ese conjunto sean el subconjunto aleatorio de tamaño r).

Debemos observar que la capacidad de generar un subconjunto aleatorio es de particular importancia en las pruebas médicas. Por ejemplo, supongamos que un centro médico planea una prueba con un nuevo medicamento diseñado para reducir el nivel de colesterol en la sangre. Para probar su eficacia, el centro médico ha reclutado 1000 voluntarios como sujetos de prueba. Para tomar en cuenta la posibilidad de que el nivel de colesterol en la sangre de los sujetos pudiera haber sido influido por factores externos (como el cambio en las condiciones climáticas), se ha decidido separarlos en dos grupos de tamaño 500: un grupo de *tratamiento* que recibe el medicamento y un grupo de *control* que recibe un placebo. Ni los voluntarios ni los que administran el medicamento sabrán quién está en cada grupo (tal prueba es *doble ciego*). Faltaría determinar cuáles de los voluntarios se deben elegir para formar el grupo de tratamiento. Es claro que uno desea que ambos grupos sean lo más similares posible en todos los aspectos, excepto porque los miembros del primer grupo recibirán el medicamento, mientras los del segundo grupo recibirán un placebo; pues en este caso sería posible concluir que cualquier diferencia de respuesta entre los grupos se debe realmente al medicamento. Hay un acuerdo general de que la mejor forma de lograrlo es elegir los 500 voluntarios del grupo de tratamiento de una manera completamente aleatoria. Es decir, la elección se debe realizar de modo que cada uno de los $\binom{1000}{500}$ subconjuntos de 500 voluntarios tenga la misma probabilidad de constituir el grupo de tratamiento. ■

Ejemplo 4c Cálculo de promedios. Suponga que queremos aproximar $\bar{a} = \sum_{i=1}^n a(i)/n$, donde n es grande y los valores $a(i)$, $i = 1, \dots, n$, son complicados y no tan fáciles de calcular. Una forma de hacerlo es observar que si X es una variable aleatoria discreta uniforme sobre los enteros $1, \dots, n$, entonces la variable aleatoria $a(X)$ tiene una media dada por

$$E[a(X)] = \sum_{i=1}^n a(i)P\{X = i\} = \sum_{i=1}^n \frac{a(i)}{n} = \bar{a}$$

Por lo tanto, si para obtener k variables aleatorias discretas uniformes X_i , $i = 1, \dots, k$, generamos k números aleatorios U_i y hacemos $X_i = \text{Ent}(nU_i) + 1$, entonces cada una de las k variables aleatorias $a(X_i)$ tendrá media $\bar{a}$, lo cual, junto con la ley fuerte de los grandes números, implica que cuando k es grande (aunque mucho menor que n), el promedio de estos valores será aproximadamente igual $\bar{a}$. Por lo tanto, podemos aproximar $\bar{a}$, utilizando

$$\bar{a} \approx \sum_{i=1}^k \frac{a(X_i)}{k}$$

Otra variable aleatoria que se puede generar sin necesidad de buscar el intervalo donde caiga el número aleatorio es la geométrica.

Ejemplo 4d Recuerde que X es una variable aleatoria geométrica con parámetro p si

$$P\{X = i\} = pq^{i-1}, \quad i \geq 1, \text{ donde } q = 1 - p$$

Se puede pensar que X representa el tiempo del primer éxito, cuando se realizan ensayos independientes, de los que cada uno es un éxito con probabilidad p . Como

$$\begin{aligned} \sum_{i=1}^{j-1} P\{X = i\} &= 1 - P\{X > j-1\} \\ &= 1 - P\{\text{primeros } j-1 \text{ ensayos sean todos fracasos}\} \\ &= 1 - q^{j-1}, \quad j \geq 1 \end{aligned}$$

podemos generar el valor de X al generar un número aleatorio U y hacer X igual al valor j para el cual

$$1 - q^{j-1} \leq U < 1 - q^j$$

o, en forma equivalente, para el cual

$$q^j < 1 - U \leq q^{j-1}$$

Es decir, podemos definir X como

$$X = \text{Mín}\{j: q^j < 1 - U\}$$

Por lo tanto, con base en que el logaritmo es una función monótona, y entonces $0 < a < b$ es equivalente a $\log^*(a) < \log^*(b)$, tenemos que X se puede expresar como

$$X = \text{Mín}\{j: j \log(q) < \log(1 - U)\} \\ = \text{Mín}\left\{j: j > \frac{\log(1 - U)}{\log(q)}\right\}$$

donde la última desigualdad cambia de signo, ya que $\log(q)$ es negativo para $0 < q < 1$. Por lo tanto, con la notación $\text{Ent}()$ podemos expresar X como

$$X = \text{Ent}\left(\frac{\log(1 - U)}{\log(q)}\right) + 1$$

Por último, al observar que $1 - U$ también está uniformemente distribuida en $(0, 1)$, tenemos que

$$X = \text{Ent}\left(\frac{\log(U)}{\log(q)}\right) + 1$$

también es geométrica con parámetro p . ■

4.2 Generación de una variable aleatoria Poisson

La variable aleatoria X es Poisson con media λ si

$$p_i = P\{X = i\} = e^{-\lambda} \frac{\lambda^i}{i!} \quad i = 0, 1, \dots$$

La clave para aplicar el método de la transformada inversa para generar esta variable aleatoria es la siguiente identidad (demostrada en la sección 2.4 del capítulo 2):

$$p_{i+1} = \frac{\lambda}{i+1} p_i, \quad i \geq 0 \quad (4.1)$$

Al aprovechar esta recursión para calcular las probabilidades Poisson conforme se necesitan, el algoritmo de la transformada inversa para generar una variable aleatoria

* $\log(x)$ representa el logaritmo natural de x . [N. del R.T.]

Poisson con media λ se puede expresar como sigue (la cantidad i se refiere al valor en cuestión; $p = p_i$ es la probabilidad de que X sea igual a i , y $F = F(i)$ es la probabilidad de que X sea menor o igual a i).

PASO 1: Generar un número aleatorio U .

PASO 2: $i = 0, p = e^{-\lambda}, F = p$.

PASO 3: Si $U < F$, hacer $X = i$ y terminar.

PASO 4: $p = \lambda p / (i + 1), F = F + p, i = i + 1$.

PASO 5: Ir al paso 3.

(Debemos observar en lo anterior que cuando escribimos, por ejemplo, $i = i + 1$, no queremos decir que i sea igual a $i + 1$, sino que el valor de i se debe incrementar en 1.) Para ver que el algoritmo anterior realmente genera una variable aleatoria Poisson con media λ , observemos que primero genera un número aleatorio U y luego verifica si $U < e^{-\lambda} = p_0$. En tal caso, hace $X = 0$. En caso contrario, en el paso 4 calcula p_1 mediante la recursión (4.1). Ahora verifica si $U < p_0 + p_1$ (donde el lado derecho es el nuevo valor de F), y en tal caso hace $X = 1$, y así sucesivamente.

El algoritmo anterior verifica en forma sucesiva si el valor de Poisson es 0, luego si es 1, luego 2, etc. Así, el número de comparaciones necesarias será 1 más que el valor generado de la Poisson. Por lo tanto, en promedio, lo anterior necesitará realizar $1 + \lambda$ búsquedas. Aunque esto está bien si λ es pequeño, se puede mejorar en gran medida si λ es grande. De hecho, como una variable aleatoria Poisson con media λ tiene una gran probabilidad de tomar uno de los dos valores enteros más cercanos a λ , un algoritmo más eficiente verificaría primero uno de estos valores, en vez de partir de 0 y trabajar hacia arriba. El algoritmo 4-1, escrito en BASIC y que aparece en el Apéndice de programas, hace $I = \text{Ent}(\lambda)$ y calcula p_I (considera primero los logaritmos y luego eleva el resultado a la potencia e). Luego utiliza la ecuación (4.1) para determinar $F(I)$ en forma recursiva. Ahora, para generar una variable aleatoria Poisson con media λ genera un número aleatorio U , y observa si $X \leq I$ revisando si $U \leq F(I)$. Luego busca hacia abajo, a partir de $X = I$ en el caso en que $X \leq I$ y hacia arriba si $X = I + 1$ (el algoritmo 4-1 también evita la posible dificultad de que, debido a errores de redondeo, cuando λ es grande la computadora podría considerar $e^{-\lambda}$ igual a cero).

Observación El número de búsquedas necesarias para el algoritmo 4-1 es aproximadamente 1 más que la diferencia absoluta entre la variable aleatoria X y su media λ . Como para λ grande una Poisson es (por el teorema central del límite) aproximadamente normal con media y varianza iguales a λ , tenemos que

Número promedio de búsquedas $\approx 1 + E[|X - \lambda|]$ donde $X \sim N(\lambda, \lambda)^\dagger$

$$\begin{aligned} &= 1 + \sqrt{\lambda} E\left[\frac{|X - \lambda|}{\sqrt{\lambda}}\right] \\ &= 1 + \sqrt{\lambda} E[|Z|] \quad \text{donde } Z \sim N(0, 1) \\ &= 1 + 0.798\sqrt{\lambda} \quad (\text{véase el ejercicio 12}) \end{aligned}$$

Es decir, al utilizar el algoritmo 4-1, el número promedio de búsquedas crece con la raíz cuadrada de λ conforme λ crece cada vez más. ■

4.3 Generación de variables aleatorias binomiales

Suponga que queremos generar el valor de X , una variable aleatoria binomial (n, p) ; es decir, X tal que

$$P\{X = i\} = \frac{n!}{i!(n-i)!} p^i (1-p)^{n-i}, \quad i = 0, 1, \dots, n$$

Para esto, nos valemos del método de la transformada inversa y de la identidad recursiva

$$P\{X = i + 1\} = \frac{n-i}{i+1} \frac{p}{1-p} P\{X = i\}$$

Si i denota el valor en cuestión, $pr = P\{X = i\}$ la probabilidad de que X sea igual a i , y $F = F(i)$ la probabilidad de que X sea menor o igual a i , el algoritmo se puede expresar de la manera siguiente:

Algoritmo de la transformada inversa para generar una variable aleatoria binomial (n, p)

PASO 1: Generar un número aleatorio U .

PASO 2: $c = p/(1-p)$, $i = 0$, $pr = (1-p)^n$, $F = pr$.

PASO 3: Si $U < F$, hacer $X = i$ y terminar.

PASO 4: $pr = [c(n-i)/(i+1)]pr$, $F = F + pr$, $i = i + 1$.

PASO 5: Ir al paso 3.

[†] Empleamos la notación $X \sim F$ para indicar que X tiene la función de distribución F . El símbolo $N(\mu, \sigma^2)$ representa una distribución normal con media μ y varianza σ^2 .

El algoritmo anterior verifica primero si $X = 0$, luego si $X = 1$, y así sucesivamente. Por lo tanto, el número de búsquedas que realiza es 1 más que el valor de X . Por consiguiente, en promedio, se necesitan $1 + np$ búsquedas para generar X . Como una variable aleatoria binomial (n, p) representa el número de éxitos en n ensayos independientes, en los que cada uno es un éxito con probabilidad p , esto implica que esta variable aleatoria también se puede generar restando a n el valor de una variable aleatoria binomial $(n, 1-p)$ (¿por qué?). Así, cuando $p > 1/2$, podemos generar una variable aleatoria binomial $(n, 1-p)$ mediante el método anterior y restar su valor de n para obtener la generación deseada.

Observaciones

1. Otra forma de generar una variable aleatoria binomial (n, p) utiliza su interpretación como el número de éxitos en n ensayos independientes, al generar n números aleatorios $U_1, \dots, U_n$ y luego hacer X igual al número de U_i que son menores o iguales a p . Si consideramos el i -ésimo ensayo como un éxito si $U_i < p$ y observamos que la probabilidad de este evento es igual a p , es fácil ver que esto produce una variable aleatoria binomial (n, p) . Sin embargo, este método requiere n números aleatorios y realiza n comparaciones, mientras que el algoritmo de la transformada inversa sólo necesita un número aleatorio y realiza, en promedio, $1 + np$ comparaciones (junto con igual número de divisiones).
2. Como en el caso Poisson, cuando la media np es grande es mejor determinar primero si el valor generado es menor o igual a $I \equiv \text{Ent}(np)$ o si es mayor que I . En el primer caso, uno debe comenzar la búsqueda con $X = I$, luego con $X = I - 1, \dots$, etc., mientras que en el segundo caso uno debe iniciar la búsqueda con $I + 1$ e ir hacia arriba. ■

4.4 La técnica de aceptación y rechazo

Suponga que tenemos un método eficiente para simular una variable aleatoria con función de masa de probabilidad $\{q_j, j \geq 0\}$. Podemos emplearlo como la base para simular a partir de la distribución que tiene función de masa $\{p_j, j \geq 0\}$ primero simulando una variable Y aleatoria con función de masa $\{q_j\}$ y luego aceptar este valor simulado con una probabilidad proporcional a p_j/q_j .

Específicamente, sea c una constante tal que

$$\frac{p_j}{q_j} \leq c \quad \text{para toda } j \text{ tal que } p_j > 0 \quad (4.2)$$

Ahora tenemos la siguiente técnica, llamada método de rechazo o método de aceptación y rechazo, para simular una variable aleatoria X con función de masa $p_j = P\{X = j\}$.

Método de rechazo

- PASO 1: Simular el valor de Y , con función de masa de probabilidad q_j .
 PASO 2: Generar un número aleatorio U .
 PASO 3: Si $U < p_j/cq_j$, hacer $X = Y$ y terminar. En caso contrario, regresar al paso 1.

El método de rechazo se representa en la figura 4.1.

Ahora demostramos que el método de rechazo funciona.

Teorema El algoritmo de aceptación y rechazo genera una variable aleatoria X tal que

$$P\{X = j\} = p_j, \quad j = 0, \dots$$

Además, el número de iteraciones del algoritmo necesarias para obtener X es una variable aleatoria geométrica con media c .

Demostración Para comenzar, determinaremos la probabilidad de que una única iteración produzca el valor aceptado j .

$$\begin{aligned} P\{Y = j, \text{ aceptada}\} &= P\{Y = j\}P\{\text{aceptada} \mid Y = j\} \\ &= q_j \frac{p_j}{cq_j} \\ &= \frac{p_j}{c} \end{aligned}$$

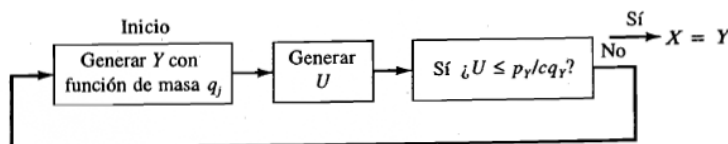


Figura 4.1 Aceptación y rechazo

Al sumar sobre j se obtiene la probabilidad de que una variable aleatoria generada sea aceptada:

$$P\{\text{aceptada}\} = \sum_j \frac{p_j}{c} = \frac{1}{c}$$

Como cada iteración produce de manera independiente un valor aceptado con probabilidad $1/c$, vemos que el número de iteraciones necesarias es una geométrica con media c . Además,

$$\begin{aligned} P\{X = j\} &= \sum_n P\{j \text{ aceptada en la iteración } n\} \\ &= \sum_n (1 - 1/c)^{n-1} \frac{p_j}{c} \\ &= p_j \end{aligned}$$

Observaciones

1. El lector debe observar que la forma en que "aceptamos el valor Y con probabilidad p_j/cq_j " consiste en generar un número aleatorio U y luego aceptar Y si $U \leq p_j/cq_j$.
2. Como cada iteración del algoritmo produce de manera independiente un valor aceptado con probabilidad $P\{\text{aceptación}\} = K = 1/c$, esto implica que el número de iteraciones necesarias tendrá una distribución geométrica con parámetro $1/c$ y por tanto tendrá media c . Entonces, mientras más cerca esté c de 1 (nunca puede ser menor que 1, lo cual se puede ver al escribir la desigualdad (4.2) como $p_j \leq cq_j$ y sumar ambos lados) más eficaz será el método; esto es claro intuitivamente, pues mientras más cerca esté c de 1, más parecidas serán las dos funciones de masa $\{p_j\}$ y $\{q_j\}$. ■

Ejemplo 4e Suponga que queremos simular el valor de una variable aleatoria X que toma uno de los valores $1, 2, \dots, 10$ con probabilidades respectivas $0.11, 0.12, 0.09, 0.08, 0.12, 0.10, 0.09, 0.09, 0.10, 0.10$. Una posibilidad es utilizar el algoritmo de la transformada inversa, pero un planteamiento mejor consiste en usar el método de rechazo, con q como la densidad uniforme discreta en $1, \dots, 10$. Es decir, $q_j = 1/10$, $j = 1, \dots, 10$. Para esta elección de $\{q_j\}$, podemos elegir c como

$$c = \text{Máx} \frac{p_j}{q_j} = 1.2$$

de modo que el algoritmo sería el siguiente:

PASO 1: Generar un número aleatorio U_1 y hacer $Y = \text{Ent}(10U_1) + 1$.

PASO 2: Generar un segundo número aleatorio U_2 .

PASO 3: Si $U_2 \leq p_Y/0.12$, hacer $X = Y$ y terminar. En caso contrario, regresar al paso 1.

La constante 0.12 del paso 3 se debe a que $cq_Y = 1.2/10 = 0.12$. En promedio, este algoritmo requiere sólo 1.2 iteraciones para obtener el valor generado de X . ■

La fuerza del método de rechazo, una de cuyas primeras versiones fue propuesta por el famoso matemático John von Neumann, será aún más clara cuando consideremos su análogo al generar variables aleatorias continuas.

4.5 El método de composición

Suponga que tenemos un método eficiente para simular el valor de una variable aleatoria con una de las dos funciones de masa de probabilidad $\{p_j^1, j \geq 0\}$ o $\{p_j^2, j \geq 0\}$ y que queremos simular el valor de la variable aleatoria X con función de masa

$$P\{X = j\} = \alpha p_j^1 + (1 - \alpha)p_j^2, \quad j \geq 0 \quad (4.3)$$

donde $0 < \alpha < 1$. Una forma de simular esta variable aleatoria X es observar que si X_1 y X_2 son variables aleatorias con funciones de masa respectivas $\{p_j^1\}$ y $\{p_j^2\}$, entonces la variable aleatoria X definida como

$$X = \begin{cases} X_1 & \text{con probabilidad } \alpha \\ X_2 & \text{con probabilidad } 1 - \alpha \end{cases}$$

tendrá su función de masa dada por (4.3). Esto implica que para generar el valor de tal variable aleatoria primero generamos un número aleatorio U y luego un valor de X_1 si $U < \alpha$ y de X_2 si $U > \alpha$.

Ejemplo 4f Suponga que queremos generar el valor de una variable aleatoria X tal que

$$p_j = P\{X = j\} = \begin{cases} 0.05 & \text{para } j = 1, 2, 3, 4, 5 \\ 0.15 & \text{para } j = 6, 7, 8, 9, 10 \end{cases}$$

Si observamos que $p_j = 0.5p_j^1 + 0.5p_j^2$, donde

$$p_j^1 = 0.1, \quad j = 1, \dots, 10 \quad \text{y} \quad p_j^2 = \begin{cases} 0 & \text{para } j = 1, 2, 3, 4, 5 \\ 0.2 & \text{para } j = 6, 7, 8, 9, 10 \end{cases}$$

para ello primero generamos un número aleatorio U y después a partir de la uniforme discreta en $1, \dots, 10$ si $U < 0.5$ y de la uniforme discreta en $6, 7, 8, 9, 10$ en caso contrario. Es decir, podemos simular X como sigue:

PASO 1: Generar un número aleatorio U_1 .

PASO 2: Generar un número aleatorio U_2 .

PASO 3: Si $U_1 < 0.5$, sea $X = \text{Ent}(10U_2) + 1$. En caso contrario, sea $X = \text{Ent}(5U_2) + 6$. ■

Si $F_i, i = 1, \dots, n$ son funciones de distribución y $\alpha_i, i = 1, \dots, n$ son números no negativos cuya suma es 1, entonces la función de distribución F dada por

$$F(x) = \sum_{i=1}^n \alpha_i F_i(x)$$

es una *mezcla*, o *composición*, de las funciones de distribución $F_i, i = 1, \dots, n$. Una forma de simular a partir de F es simular una variable aleatoria I , igual a i con probabilidad $\alpha_i, i = 1, \dots, n$ y luego simular a partir de la distribución F_i (es decir, si el valor simulado de I es $I = j$, entonces la segunda simulación es a partir de F_j). Este método para simular a partir de F se conoce con frecuencia como el *método de composición*.

Ejercicios

1. Escriba un programa para generar n valores a partir de la función de masa de probabilidad $p_1 = 1/3, p_2 = 2/3$.
 - (a) Sea $n = 100$, ejecute el programa y determine la proporción de valores que sean iguales a 1.
 - (b) Repita (a) con $n = 1000$.
 - (c) Repita (a) con $n = 10000$.
2. Escriba un programa de computadora tal que, dada una función de masa de probabilidad $\{p_j, j = 1, \dots, n\}$ como entrada, proporcione como salida el valor de una variable aleatoria con esta función de masa.
3. Dé un algoritmo eficiente para simular el valor de una variable aleatoria X tal que

$$P\{X = 1\} = 0.3, \quad P\{X = 2\} = 0.2, \quad P\{X = 3\} = 0.35, \quad P\{X = 4\} = 0.15$$
4. Se baraja un conjunto de 100 cartas (numeradas del 1 al 100) y luego se voltean, una a la vez. Decimos que ocurre un "éxito" si la carta i es la i -ésima carta

volteada, $i = 1, \dots, 100$. Escriba un programa de simulación para estimar la esperanza y la varianza del número total de éxitos. Ejecute el programa. Determine las respuestas exactas y compárelas con sus estimaciones.

5. Otro método para generar una permutación aleatoria, diferente del presentado en el ejemplo 4b, consiste en generar de manera sucesiva una permutación aleatoria de los elementos $1, 2, \dots, n$ comenzando con $n = 1$, luego $n = 2$ y así sucesivamente (por supuesto, la permutación aleatoria cuando $n = 1$ es 1). Una vez que se tiene una permutación aleatoria de los primeros $n - 1$ elementos (digamos, $P_1, \dots, P_{n-1}$), para obtener la permutación aleatoria de los n elementos $1, \dots, n$ colocamos n en la posición final (para obtener la permutación $P_1, \dots, P_{n-1}, n$) e intercambiamos los elementos de la posición n (a saber, n) con el elemento de una posición elegida al azar; la posición 1, la posición 2, $\dots$, la posición n son igualmente probables.
 - (a) Escriba un algoritmo que realice lo anterior.
 - (b) Demuestre mediante inducción matemática sobre n que el algoritmo funciona, en el sentido de que la permutación obtenida tiene la misma probabilidad de ser cualquiera de las $n!$ permutaciones de $1, 2, \dots, n$.
6. (a) Emplee 100 números aleatorios para explicar la forma de determinar una aproximación de $\sum_{i=1}^N e^{i/N}$, donde $N = 10000$.
 - (b) Obtenga la aproximación.
 - (c) ¿Es buena su aproximación?
7. Se lanza de manera continua un par de dados legales, hasta que todos los posibles resultados $2, 3, \dots, 12$ hayan aparecido al menos una vez. Desarrolle un estudio de simulación para estimar el número esperado de lanzamientos necesarios.
8. Suponga que cada miembro en una lista de n elementos tiene un valor asociado, y sea $v(i)$ el valor asociado al i -ésimo elemento de la lista. Suponga que n es muy grande y también que cada elemento puede aparecer en lugares distintos de la lista. Explique la forma de utilizar números aleatorios para estimar la suma de los valores de los elementos distintos en la lista (en la que el valor de cada elemento se cuenta una sola vez, sin importar el número de veces que el elemento aparezca anotado).
9. Considere los eventos $A_1, \dots, A_n$ donde los A_i , $i = 1, \dots, n$ consisten en los siguientes n_i resultados: $A_i = \{a_{i,1}, a_{i,2}, \dots, a_{i,n_i}\}$. Suponga que para cualquier resultado dado a , se conoce $P\{a\}$, la probabilidad de que el experimento produzca el resultado a . Explique la forma de utilizar los resultados del ejercicio 8 para estimar $P(\cup_{i=1}^n A_i)$, la probabilidad de que al menos uno de los eventos A_i ocurra. Observe que no suponemos que los eventos A_i , $i = 1, \dots, n$ sean mutuamente excluyentes.

10. La función de masa de probabilidad binomial negativa con parámetros (r, p) , donde r es un entero positivo y $0 < p < 1$, está dada por

$$p_j = \frac{(j-1)!}{(j-r)!(r-1)!} p^r (1-p)^{j-r}, \quad j = r, r+1, \dots$$

- (a) Por medio de la relación entre las variables aleatorias binomial negativa y geométrica y los resultados del ejemplo 4d obtenga un algoritmo de simulación de esta distribución.
- (b) Verifique la relación

$$p_{j+1} = \frac{j(1-p)}{j+1-r} p_j$$

- (c) Utilice la relación de la parte (b) para dar otro algoritmo que genere variables aleatorias binomiales negativas.
 - (d) Utilice la interpretación de la distribución binomial negativa como el número de ensayos necesarios para acumular un total de r éxitos, cuando cada ensayo tiene éxito de manera independiente con probabilidad p , para obtener otro método para generar tal variable aleatoria.
11. Si Z es una variable aleatoria normal estándar, muestre que

$$E[|Z|] = \left(\frac{2}{\pi}\right)^{1/2} \approx 0.798$$

12. Dé dos métodos para generar una variable aleatoria X tal que

$$P\{X = i\} = \frac{e^{-\lambda} \lambda^i / i!}{\sum_{j=0}^k e^{-\lambda} \lambda^j / j!}, \quad i = 0, \dots, k$$

13. Sea X una variable aleatoria binomial con parámetros n y p . Suponga que queremos generar una variable aleatoria Y cuya función de masa de probabilidad es igual a la función de masa condicional de X dado que $X \geq k$ para alguna $k \leq n$. Sea $\alpha = P\{X \geq k\}$ y suponga que se ha calculado el valor de α .
 - (a) Dé el método de transformada inversa para generar Y .
 - (b) Dé otro método para generar Y .
 - (c) ¿Para qué valores de α , pequeños o grandes, sería ineficiente el algoritmo en (b)?
14. Dé un método de simulación a partir de la función de masa de probabilidad p_j , $j = 5, 6, \dots, 14$, donde

$$p_j = \begin{cases} 0.11 & \text{cuando } j \text{ es impar y } 5 \leq j \leq 13 \\ 0.09 & \text{cuando } j \text{ es par y } 5 \leq j \leq 14 \end{cases}$$

15. Suponga que la variable aleatoria X puede tomar cualquiera de los valores $1, \dots, 10$ con probabilidades respectivas $0.06, 0.06, 0.06, 0.06, 0.06, 0.15, 0.13, 0.14, 0.15, 0.13$. Utilice el método de composición para dar un algoritmo que genere el valor de X .
16. Presente un método para generar el valor de X , en el que

$$P\{X = j\} = \left(\frac{1}{2}\right)^{j+1} + \frac{\left(\frac{1}{2}\right)^{2j-1}}{3^j}, \quad j = 1, 2, \dots$$

17. Sea X con función de masa $p_j = P\{X = j\}$, $\sum_{j=1}^{\infty} p_j = 1$. Sea

$$\lambda_n = P\{X = n | X > n-1\} = \frac{p_n}{1 - \sum_{j=1}^{n-1} p_j}, \quad n = 1, \dots$$

- (a) Muestre que $p_1 = \lambda_1$ y

$$p_n = (1 - \lambda_1)(1 - \lambda_2) \cdots (1 - \lambda_{n-1})\lambda_n$$

Las cantidades λ_n , $n \geq 1$, son las tasas discretas de riesgo, pues si pensamos en X como el tiempo de vida de algún artículo, entonces λ_n representa la probabilidad de que alguno que haya alcanzado la edad n muera durante ese periodo. El siguiente método para simular variables aleatorias discretas, llamado método de la tasa discreta de riesgo, genera una sucesión de números aleatorios y termina cuando el n -ésimo número aleatorio es menor que λ_n . El algoritmo se puede escribir como sigue:

PASO 1: $X = 1$.

PASO 2: Generar un número aleatorio U .

PASO 3: Si $U < \lambda_X$, terminar.

PASO 4: $X = X + 1$.

PASO 5: Ir al paso 2.

- (b) Muestre que el valor de X al terminar este proceso tiene la función de masa deseada.
- (c) Suponga que X es una variable aleatoria geométrica con parámetro p . Determine los valores λ_n , $n \geq 1$. Explique lo que hace el algoritmo anterior en este caso y por qué es evidente su validez.
18. Suponga que $0 \leq \lambda_n \leq \lambda$, para toda $n \geq 1$. Considere el siguiente algoritmo para generar una variable aleatoria con tasas discretas de riesgo $\{\lambda_n\}$.

PASO 1: $S = 0$.

PASO 2: Generar U y hacer $Y = \text{Ent}\left(\frac{\log(U)}{\log(1 - \lambda)}\right) + 1$.

PASO 3: $S = S + Y$.

PASO 4: Generar U .

PASO 5: Si $U \leq \lambda_S/\lambda$, hacer $X = S$ y terminar. En caso contrario, ir a 2.

(a) ¿Cuál es la distribución de Y en el paso 2?

(b) Explique lo que hace el algoritmo.

(c) Argumente que X es una variable aleatoria con tasas discretas de riesgo $\{\lambda_n\}$.